

Assignment 5

Composing Functions Using Pipelining

Introduction to Functional Programming | Academic year 2026 2027

Maximum score 15 points | Individual work or teams of 2 to 4 students

Implement one complete data processing workflow by composing small pure functions. This assignment should become the first working part of your final project, with configurable rules and a clear separation between calculations and input or output.

Use your planned final project or a reusable module that will be integrated into it. Keep one runnable project across Assignments 5 to 8. A console interface is sufficient; a new application or a graphical interface is not required for each assignment.

Every team member must implement meaningful logic, contribute automated tests, and defend their own work. Agree on responsibilities before coding. Earlier relevant code and tests may be reused and improved.

Task 1 Define the workflow

Choose a useful operation in your project: calculate delivery quotes, score applications, select daily tasks, or produce a budget summary. In README.md, state the topic, intended users, planned team, input fields, business rules, and expected result. Implement one operation completely before expanding the project.

Prepare a small dataset with at least five records. Include records that should be accepted and rejected, a boundary value, and data that produces different results under your two policies. Document expected results before running the program.

Task 2 Define inputs and validation

Create typed input and output models. Prefer records, readonly record structs, or other models with immutable values. A collection exposed as IReadOnlyList is not automatically an immutable copy; avoid sharing mutable collections that callers can later change.

Parse and validate at the application boundary. Distinguish missing input from a valid empty dataset and from legitimate values such as zero or false. Reject invalid fields with a clear message. Do not silently replace invalid values with defaults or suppress a missing-value warning with the null-forgiving operator.

For this stage, ordinary validation or a TryParse style approach is sufficient. Result and Maybe are introduced in Assignment 7. Keep valid domain data separate from raw text.

Pipeline implementation

Task 3 Compose at least three pure stages

Create at least three named transformation functions, and connect them in one visible application workflow. Each stage must do useful domain work and pass its result to the next stage. Document the input type, output type, rule, and a concrete example for every stage.

- Stage 1 selects or normalises validated records according to a domain rule.
- Stage 2 computes a score, cost, priority, or another domain value for the selected records.
- Stage 3 creates a result model, such as ordered recommendations, quotes, or a summary.

These roles are examples; choose stages that fit your project. A function that merely forwards its argument and a function that prints to Console do not count as transformation stages. Composition may use ordinary function calls, Func delegates, or a small composition helper; a generic pipeline framework is not required.

Task 4 Make one rule configurable

Implement at least one higher-order function that accepts a Func or another delegate and actually calls it during processing. Provide two policies for the same rule. Select the policy outside the pure pipeline and pass it as an argument; do not hide the selection in a mutable global field.

Run both policies on the same dataset. Show a case where the result changes because the rule changes, while the pipeline implementation stays the same. Captured configuration must stay stable during the workflow; do not capture a mutable global setting or modify captured values during processing.

Task 5 Keep the functional core pure

The same input and policy must produce the same output. Pure stages must not modify caller-owned objects or collections, read Console input, write files, or depend on an unstated clock, random value, or mutable global state. Pass any required external value explicitly. Perform display and file operations after the result has been computed.

Example of an acceptable workflow

A delivery project can select valid requests, calculate costs using a supplied price policy, and produce quote records. For demonstration, a standard policy might be 500 plus 100 per kilometre and an express policy 800 plus 150 per kilometre. For a valid 2 kilometre request, the documented results are 700 and 1100 respectively. An invalid negative distance is rejected before the calculation. Adapt the workflow and policies to your own project; these exact prices are not required.

Tests and individual defense

Provide at least 6 cumulative automated tests. Use an assertions-based test framework or a test runner that compares actual and expected values and reports failures. Console output alone is not a test. Relevant earlier tests count toward the minimum.

The cases below define required coverage. One test may contain several assertions, and one scenario may cover several requirements. Use additional tests if needed; the minimum number does not remove any required case.

CASE	REQUIRED ASSERTION
NORMAL WORKFLOW	A valid dataset passes through the composed stages and produces the documented result.
EMPTY DATASET	A valid empty dataset produces the documented empty result or summary without crashing.
BOUNDARY VALUE	An exact threshold or minimum value follows the stated inclusion and calculation rule.
INVALID AND MISSING INPUT	Malformed fields and null or missing required input are rejected; no fabricated successful result is returned.
TWO POLICIES	Both policy results are checked on the same input and differ in the documented case.
PURITY AND STAGES	Check an individual stage and assert that caller-owned input values and collections remain unchanged.

What to show during the defense

- Run the complete workflow with both policies and show the expected output.
- Trace one record through all stages and explain each input and output type.
- Identify the higher-order function and the delegate invocation. Explain why the pure stages have no hidden side effects.
- Show invalid input and an empty dataset, then run the tests.
- Complete a small requested change, such as modifying a policy or a threshold, and update or run a relevant assertion.

Assessment and submission

Assessment

CRITERION	EVIDENCE ASSESSED	POINTS
WORKFLOW CORRECTNESS	The documented domain operation produces correct normal and edge results.	1
PIPELINE AND COMPOSITION	At least three useful pure stages with compatible contracts are composed and used.	3
FUNCTIONAL SEPARATION	Stable inputs, no mutation of caller-owned data or hidden mutable state, and isolated input and output.	2
CONFIGURABLE BEHAVIOUR	A higher-order function invokes one of two working policies.	1
TESTS AND DOCUMENTATION	At least 6 automated tests cover the required cases; stages and contributions are documented.	1
INDIVIDUAL DEFENSE	Explain your own pipeline and complete a small live modification.	7
TOTAL		15

The implementation criteria total 8 points; the individual defense is worth 7 points. Each student receives an individual grade. Only implemented constructs used by the application earn points. A student who does not defend receives 0. If you cannot explain your solution, it is not accepted under the course code defense policy. Academic integrity and AI rules follow the syllabus.

What to upload

Every student must upload a ZIP archive with a runnable snapshot for Assignment 5. Team members may upload the same project code, but each must identify and defend their own contribution. Use the deadline and submission policy published in Moodle.

- Include C# source, project and solution files, automated tests, README.md, and sample input files needed to run the demonstrated workflows. Exclude generated .NET bin and obj folders, IDE caches, and local secrets.
- State the .NET SDK version, exact build, run and test commands, expected working directory, and any prerequisites. Check these instructions from a clean extraction of the archive.

README content

- Describe the project workflow, its input, domain rules, expected output, and short instructions for the normal and failure demonstrations.
- Map each requirement to the files and methods that implement it. List test cases, expected results, and the latest test outcome.
- List each member by name and group and explain their implemented logic, tests, and documentation for this stage. Team size does not increase the number of required workflows or reports.

Use in the final project

Retain the code and tests for the next stage. The final project must be hosted in an accessible Git repository on GitHub, GitLab, or another Git hosting service. Repository links do not replace the files required in Moodle. The final project has a separate assessment of 50 points for the project and 50 points for the individual defense, with a group project report and an individual contribution report.

Optional introduction to Git

Git use is optional for Assignment 5 and does not earn or remove rubric points. Hosting becomes mandatory for the final project. Git records versions locally; a commit saves selected changes. A remote repository stores a shared copy online. A branch separates feature work before it is merged.

Install Git, configure your own name and email, and create an empty repository on your chosen hosting service. In the project folder, create a .gitignore for bin/, obj/, .vs/, .idea/, and local secrets. Replace the example identity and REPOSITORY_URL below; authenticate using the hosting service instructions.

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
git init
git add .
git status
git commit -m "Add initial project pipeline"
git branch -M main
git remote add origin REPOSITORY_URL
git push -u origin main
```

One team member creates the repository and invites the others. Teammates use git clone REPOSITORY_URL once. Start from a clean working tree, update main, create a feature branch, implement and test a change, then commit and push it.

```
git switch main
git pull
git switch -c feature/pipeline
git add .
git status
git commit -m "Implement pipeline stages and tests"
git push -u origin feature/pipeline
```

Open a pull request or merge request, review and merge the tested change, then return to main and pull. Resolve conflicts by checking the affected lines, testing, and committing the resolution. Never upload passwords or tokens. Official guide: <https://git-scm.com/book/en/v2>